

From Text to Diagnosis:

Evaluating GPT-Based Models for Classifying Depression Severity in Online Texts

Exam Paper for:
Natural Language Processing & Text Analytics by



Michael Wong (163841)

Normal Pages: 15, Number of Characters: 34.146

MSc in Business Administration and Data Science
The Faculty of Digitalization

Copenhagen, 7th of August 2024

Abstract

This paper investigates the effectiveness of various prompt engineering techniques and retrieval augmented generation in classifying depression levels from Reddit posts using large language models (GPT-3.5 Turbo & GPT-4o). Dreaddit and DepSeverity datasets were combined to evaluate the effectiveness of: zero-shot, few-shot, chain-of-thought prompting, and DSPy, a Python framework for automated prompt optimization. DSPy was the most effective method increasing model performance (macro F1) by 46,2% and reducing model error (RMSE) by 29,7%. Few-shot prompting increased performance by 26,5% and reduced error by 17,2%. The effectiveness of chain-of-thought and retrieval augmented generation were mixed. Despite the overall increase in performance, even the best performing LLM method showed results only marginally better compared to a traditional non-semantic method such as logistic regression.

Part I - Introduction

Depression is a serious mental health disorder that can afflict anyone and significantly reduce their quality of life. The disease can affect children and adults alike, causing mood and behavioural changes that affect a person's emotional, social, and physical health (National Institute of Mental Health, 2024). As of 2021, the number of depression patients globally reached an estimated 4% of the population (IHME, 2024). This presents a big challenge, necessitating increased healthcare expenditure to accommodate the large number of patients. In Denmark, approximately 637 thousand people or 13.2% of the total population suffered from depression in 2021 (Rosendahl, H., et al., 2022), with the costs of treating major depressive disorder estimated at 5.5 billion DKK (Christensen et al., 2022).

One way this can be tackled is with the early detection and accurate classification of depression levels among patients. Namely, natural language processing (NLP) can be used to automate language analysis to gain a glimpse into the mental state of the patient. By mapping the severity of depressive thoughts expressed in the patients' text onto a scale, the depression level can be categorized according to severity. This can lead to rapid identification of severe cases and prioritize them for treatment.

Ongoing research into large language models (LLMs) has led to the release of numerous models and advanced their applications in classification tasks. Namely, commercially available models such as GPT 3.5 have been used on text classification tasks such as stress, depression, and suicide detection (Lamichhane, 2023; Amin et al., 2023). Other research has explored GPT-3.5's reasoning capabilities such as identifying potential stressors in the mental health context (Yang et al., 2023).

Moreover, Xu et al. (2024) evaluated several LLMs, including Alpaca, Alpaca-LoRA, FLAN-T5, LLaMA2, GPT-3.5, and GPT-4, on several mental health classification tasks. In addition to evaluating the performance of various models, and fine tuning them, they also examined the performance of zero-shot prompting, few-shot prompting, and instruction fine tuning on the LLMs' performance. Their fine-tuned models, such as Mental-Alpaca and Mental-FLAN-T5 reached performance in line with leading task-specific models such as Mental-RoBERTa. Notably, both GPT-3.5 and GPT-4 achieved worse balanced classification accuracy results with the zero-shot and few-shot prompts. Our research builds upon Xu et al. (2024) by attempting to improve the performance of their applied methods and modifying the methodology to include chain-of-thought (CoT) and retrieval augmented generation (RAG), as well as automatic prompt engineering using DSPy. Moreover, we explore the capabilities of GPT 4o, and finally compare the performance of all the models with traditional classification approaches such as Naive Bayes (NB) and Logistic Regression (LR).

Part II - Data

2.1 Description & Analysis

Example Text: I have this feeling of dread about school right before I go to bed and I wake up with an upset stomach which lasts all day and makes me feel like I'll throw up. This causes me to lose appetite and not wanting to drink water for fear of throwing up. I'm not sure where else to go with this, but I need help. If any of you have this, can you tell me how you deal with it? I'm tired of having this every day and feeling like I'll throw up.

Label: Moderate depression (2)

Figure 1 - Example of Reddit post from Dataset with Attached Depression Severity Level

The dataset used in this paper was created by combining the Dreaddit (Turkan & McKeown, 2019) and DepSeverity (Naseem et al., 2022) datasets. Dreaddit is a corpus of 187,444 scraped Reddit¹ posts from 10 different subreddits. From this corpus, due to labelled data availability, this paper investigates approximately 3000 of the posts. The largest of these posts were divided into smaller parts, resulting in 3554 segments² that form the building blocks of the analysis. DepSeverity is a dataset that includes four depression severity levels for each of these 3554 segments which were annotated by two human annotators who applied the clinical depression standards. Due to the wide range of Subreddits from which the posts were scraped, there are multiple expressions of depression that are captured in the data. The variety broadens the scope of the problem to allow for classification of a broader range of depressive experiences, but also enlarges the semantic space, thus increasing the analysis complexity.

Table 1- Data Overview

Theme	Domain	Subreddits	Labeled Segments (Posts)	Average Characters
Interpersonal Conflict	Abuse	r/domesticviolence	388	433
		r/survivorsofabuse	315	476
	Social	r/relationships	694	460
Mental Illness	Anxiety	r/anxiety	650	448
		r/stress	78	527
	PTSD	r/PTSD	711	435
Financial Need	Financial Need	r/almosthomeless	99	456
		r/assistance	355	426
		r/foodpantry	43	387
		r/homeless	220	439

¹ Reddit is a social media platform where users create posts in topic-specific communities called subreddits, with other users voting and commenting on them.

² Henceforth, for simplicity, the term *posts* will be used instead of *segments*; note that our analysis does not account for the fact that some segments are from the same post.

The depression levels are calculated by rating the frequency and/or intensity of specific depressive symptoms over a set period of time (e.g. past week or recently), based on their presence in the text. After summing the scores for each symptom, the total score is converted into a t-score categorising depression severity into following levels: Minimal, Mild, Moderate, Severe.³ Note that minimal clinical depression does not imply a complete lack of symptoms - on the contrary, someone often feeling unhappy and sad, may not be clinically depressed. Therefore, the possible conflict between the clinical perception of depression, captured in the labels, and the common perception, embedded within GPT, should be considered.

The distribution of the depression severity in the dataset is as follows:

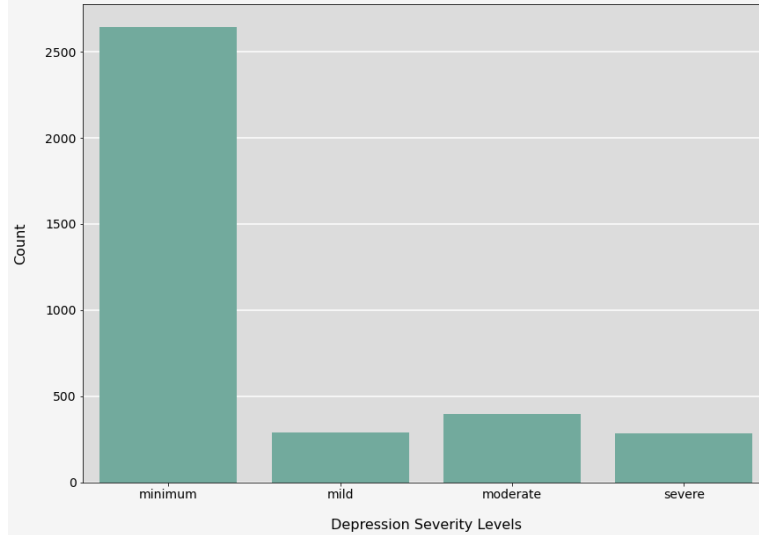


Figure 2 - Distribution of Depression Severity Labels

The data are imbalanced⁴ with minimal depression outnumbering each of the other three minority classes by about 8 times on average. Among the minority classes, mild and severe have about the same population, while moderate is approximately 40% larger than the other two. Such imbalance may introduce bias during model training and complicate the assessment of test results. With regards to training, since LLMs are pretrained, they are generally robust to data imbalance, in addition to us not exposing them directly to the entirety of the dataset; as for the traditional models, the selected solution was undersampling. This was due to: (a) not wanting to introduce synthetic samples into the data which could potentially dilute the human-created posts and, (b) manually labelling would be problematic both due to the lack of knowledge about the exact methodology, and the significant time investment required. A thorough discussion regarding the effects of imbalance on performance metric assessment is included in [Part IV - Results](#).

Lengthwise posts are on average 448 characters long with the relatively long nature of Reddit posts making them an excellent source for studying the nuances of psychological phenomena (Turkan & McKeown, 2019). To assess the data in terms of linguistic properties, the Linguistic Inquiry and Word

³ Note that the exact methodology for the annotation process is not published in the paper, while, contacting the writers of the paper proved unfruitful. We have, however, conducted a bibliographic review of similar processes to understand the employed standard practices.

⁴ Moderately, by Google's [definition](#), with each minority class comprising of 8% to 11% of the data.

Count (LIWC) was used. LIWC is a lexicon-based tool providing scores in categories relevant for psychology, such as anxiety or cognitive processes, serving as an indicator for topic prevalence and expression variety (Pennebaker et al., 2015). By studying the selected metrics (see Table 2), it becomes apparent that the use of 1st person singular pronouns (liwc_i) increases with depression. Words signalling positive (liwc_posemo) and negative emotions (liwc_negemo), as well as those suggesting anxiety (liwc_anxiety) or social processes (liwc_social), are good indicators to distinguish between minimal and more severe depression, but, among the higher than minimal levels, they are not an effective depression severity indicator. Finally, two readability indices were used to assess the text structurally; the Flesch–Kincaid grade level and the automated readability index (ARI).⁵ On average, as someone would expect from social media posts, the readability is high, using mid-elementary school language and generally, the greater the depression level, the higher the readability, at least syntactically.

Table 2 - LIWC & Syntactic Complexity by Depression Severity

Depression Severity Level	liwc_i	liwc_posemo	liwc_negemo	liwc_anxiety	liwc_social	syntax_fk_grade	syntax_ari
Minimum	8,17%	2,94%	2,66%	0,67%	12,01%	5,54	4,87
Mild	10,91%	2,17%	5,24%	1,41%	8,22%	5,68	4,82
Moderate	11,55%	1,92%	5,09%	1,82%	6,80%	5,53	4,62
Severe	11,73%	2,10%	4,81%	1,56%	7,81%	4,96	3,86
Average	9,04%	2,70%	3,31%	0,92%	10,81%	5,51	4,76

2.2 Processing

This section details key parts of the preprocessing processes such as the splitting of the data, text preprocessing steps for implementing NB and LR, as well as the code processing method to ensure that the LLMs produce the correct format.

2.2.1 Splitting the data

Besides evaluating ChatGPT models, which are already pretrained, we want to evaluate DSPy and traditional methods (NB and LR). Both require a training function, in an addition to RAG which requires a retrieval database. Therefore, we performed a stratified split of the data into 80/10/10 [training/validation/test]. The strata used for the split was both the domain and depression levels to ensure proper representation.

2.2.2 Text pre-processing

Due to the nature of bag-of-words (BoW) and TF-IDF, which relies on word frequencies, text pre-processing is recommended for NB and LR. Because the textual data is sourced from Reddit, and pre-processing standardizes the text and removes noise such as irrelevant symbols and frequent but insignificant words. For our case, we converted all texts to lowercase and removed special characters by replacing non-word characters with spaces. We also removed stopwords, and we performed lemmatization, which is the process of removing inflection of words and returning to their base form. We further investigated whether pre-processing positively impacted GPT models’ performance by applying the pre-processing steps to the validation set and evaluating the output. The results indicated that pre-processing lowered the models’ performances. Therefore, we decided to proceed without pre-processing for the subsequent GPT testing.

⁵ Both are affected by the average number of words per sentence; in f-k syllables per word are included in its calculation, whereas in ARI characters per word are used.

2.2.3 Batch and JSONL format

Exploring OpenAI’s Batch API (see 3.1.1) requires preparation. Specifically, the requests need to be structured in JSONL format. To achieve this, we defined a function to create a JSONL file, which parses parameters such as the prompt type and the specific GPT model, and we created another function to verify its content. Then a function was created to upload the file to OpenAI.

Part III – Methodology

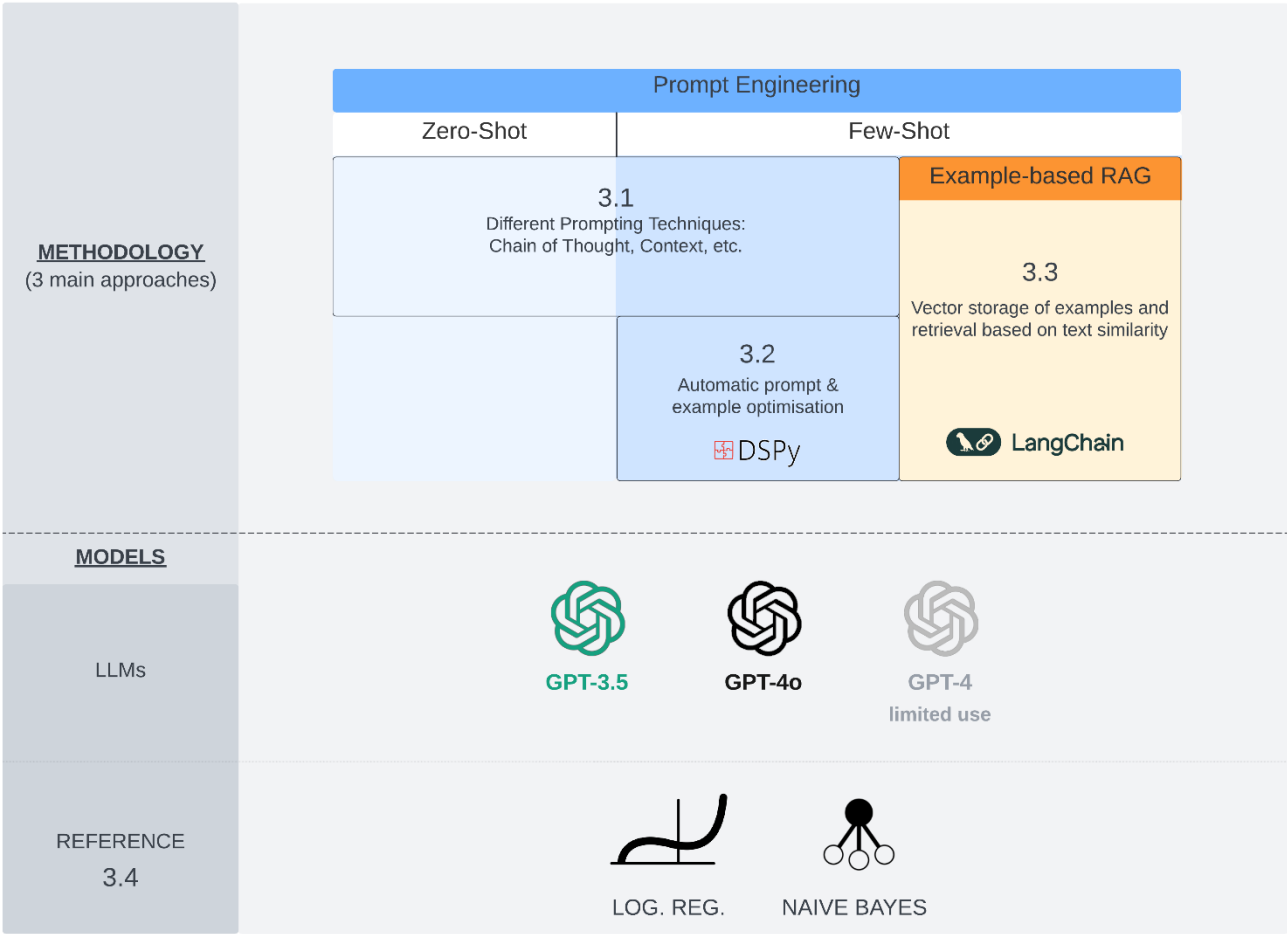


Figure 3 - Methodology Overview

3.1 Zero-Shot & Few-Shot Models

The first section introduces the LLMs used in this paper and the reasoning behind their use, while the second section details the prompt engineering and prompt selection process.

3.1.1 GPT 3.5 Turbo and GPT 4o

For the text classification we used OpenAI’s GPT 3.5 Turbo, and GPT 4o. These models are pretrained on vast amounts of text corpora across diverse domains, providing them extensive world knowledge and the ability to process discriminative tasks such as text classification (Radford et al., 2018). Notably, this extensive training allows GPTs to perform classification tasks effectively across a broad domain such as

mental health without necessitating finetuning as opposed to task-specific models that are trained on data with a narrow focus. Another advantage of these models is that they can handle natural language text data without requiring any preprocessing.

The performance of GPT 3.5 Turbo has been validated in previous research (Xu et al., 2024), providing a good low-cost baseline to measure the effects of our prompt engineering. Notably, this model allows for requesting JSON structured classification outputs, and improved performance in adhering to the required output over previous versions. This is relevant since the JSON format provides a structured representation and facilitates convenient extraction of the GPT responses, allowing us to evaluate the output of the models. However, in some cases, the model does not provide JSON format and/or numerical values. These issues can lead to costly errors and complexity with the JSON extractor, because (a) API usage incurs cost, (b) it requires elaborate error handling, and (c) it can take several iterations for the prompt produce the correct output. The context window of GPT 3.5 is 16,385 tokens, with an output of a maximum of 4,096 tokens (OpenAI, n.d.). This is sufficient capacity for processing the few-shot and RAG examples, as well as rationale and chain-of-thought prompts. GPT 4o, is a significant advancement over 3.5 Turbo, with improved metrics across all benchmarks, which might result in improved classification performance. It also has a significantly larger context window at 128K tokens and an output of 64K tokens (OpenAI, n.d.).

To access the models, we used the OpenAI API, which allows users to make requests with immediate responses or batching the requests into asynchronous groups with a 24-hour turnaround time, at a 50% lower cost and higher rate limit to real-time responses. Due to the monetary constraints of the project, and high processing needs, we tested both methods and opted for the batch method to produce the final results. The batch and real-time response capabilities of the API allow for two practical applications. Namely, in cases where the classification of the depression level is urgent (for example a suicide risk assessment tool) we can use the synchronous version of the code to immediately receive an output for the specific case. On the other hand, in the case where many classifications need to be done, and the task can be delayed by up to 24 hours (for example a tool to monitor the mental health of workers) the batch code can be used to reduce costs.

To determine the optimal performance of the GPT models, we experimented with the temperature parameter. Lower temperature increases the probability of the most probable tokens, which in our case is the depression level, while simultaneously decreasing the probability of the rare tokens (Jurafsky, D., & Martin, J. H. (2023), pp. 223–249). The reasoning behind this was to see if the increased temperature can lead to the model capturing more diverse contexts in the text data by increasing the randomness of the output. Accordingly, we tested the temperature with a value of 0.0, 0.2, and 0.4, with 0.2 providing the best metrics in the test prompt, after which it was applied to all the final tests. The temperature was tested on a single prompt due to cost constraints, with the assumption that the performance of 0.2 is superior in all other prompt types.

Table 3 - Performance by Temperature

Metric	Temperature		
	0.0	0.2	0.4
Accuracy	38,23%	39,89%	39,06%
Precision	0,395	0,389	0,385
Recall	0,414	0,415	0,404
F1	0,329	0,332	0,325
MAE	0,914	0,900	0,898
RMSE	1,261	1,260	1,250

3.1.2 Prompt Engineering

A key aspect of LLMs' performance is the prompt, an input piece of text that conditions or guides the generation of response by the model. GPTs have an increasing capability to handle long contexts which makes them powerful for conditional generation tasks by utilizing extensive portions of the initial prompt (Jurafsky, D., & Martin, J. H. (2023), pp. 223–249). As such, the way a prompt is constructed has a direct influence on the output of the language model.

The process of creating task-specific instructions to guide the output of the model is referred to as prompt engineering (Sahoo et al., 2024). Following an iterative approach, we constructed and tested several prompts, with the main techniques being zero-shot, few-shot, and chain-of-thought prompting as defined Sahoo et al. (2024). Our initial prompt design was adapted from Xu et al. (2024), with later iterations diverging significantly. As per Xu et al. (2024), we structured our prompts to consist of a Part1-S, Part2-Q, and an OutputConstraint. Part1-S refers to the instructions provided as input to the model via the prompt. There are several sets of instructions to measure the difference in performance based on the additional context. Part2-Q refers to the question or instruction that the LLM needs to answer, which in our case is ranking the depression level in the text. The OutputConstraint serves to constrain the models' output to only provide the label in a JSON format. The OutputConstraint and Part2-Q remained the same throughout the various iterations.

To measure the baseline performance of the model, we used a basic zero-shot prompt as defined by Sahoo et al (2024). This prompt only gave a direction with no labelled data or specific steps it needs to take to complete the task. As such, we can measure the model's ability to classify the text solely on its pre-existing knowledge.

We also measured the effect of additional context by constructing a more instructionally rich prompt in a zero-shot format, with iterations changing the (a) role of a psychologist and (b) source of text (see Enhancement in [Table 4](#)). The aim of the enhancement is to see if additional context in the prompt body will improve the performance of the model: (a) The role of psychologist is provided by adding “*Assume the role of a psychologist*” to the prompt. The goal of this enhancement is to see if specifying the role of psychologist can improve performance based on the assumption that doing so can make the attention layers of the model shift their focus and align the semantic context more closely with topics relevant to psychology and mental health. (b) On the other hand, the context of the text introduced as “*You will be given text data from a user's Reddit posts.*” provides the model with context of the social media origin of the text. Based on these tests, we developed a prompt that utilizes both the psychologist role and context of Reddit posts, as well as detailed instructions on the task.

Regarding few-shot prompting, we tested the performance of one handpicked selected example per class, which was included in the prompt. The goal of the examples is to help the model better understand the task by showing the desired output for a specific input, as per Sahoo et al., (2024). The final prompt was an adaptation of the enhanced zero shot prompt, containing the same role and expanded, with the additional examples, context.

Finally, we tested CoT on few-shot and zero-shot models with the aim of improving model performance by guiding them through a step-by-step reasoning process (Sahoo et al., 2024). This prompt type was also tested in several formats, with modification of examples, context, and role.

(For an overview of our additional testing techniques see [Appendix C](#)).

Table 4 - Prompt Engineering

	Baseline	Zero-Shot		Few-Shot		
		No CoT	CoT	No CoT	CoT	RAG
Enhancement	None	Context and Role	CoT, Context and Role	Context and Role	CoT, Context and Role	Context and Role
Logic	Simple zero-shot prompt with only direction to classify text.	Zero-shot prompt with detailed instructions and role of a psychologist and text context.	Zero-shot CoT prompt guiding through step-by-step reasoning process.	Few-shot prompt with detailed instructions, role of a psychologist, text context, and examples.	Few-shot CoT prompt with examples to guide through reasoning process.	Few-shot prompt with detailed instructions, role of a psychologist, text context, and examples.
PromptPart1-S		You will be given text data from a user's Reddit posts. Assume the role of a psychologist and classify the depression level of the user based on the text from the patient's Reddit posts. The depression level is on a scale of Minimum to Severe, where Minimum is no to minimal signs of depression, Mild is mild signs of depression, Moderate is moderate signs of depression, and Severe is very depressed.	Instruction: You are a psychologist analyzing text data from a user's reddit posts to determine their depression level. Classify the depression level of the user based on the text provided. The depression level is on a scale of Minimum to Severe, where: Minimum = No to minimal signs of depression Mild = Mild signs of depression Moderate = Moderate signs of depression Severe = Severe signs of depression Use the following chain of thought to analyze the text: 1. Identify any explicit statements related to feelings of sadness, hopelessness, failure, or unhappiness. 2. Assess the frequency and intensity of these feelings based on the user's descriptions. 3. Consider any additional context that might indicate a higher or lower depression level. 4. Based on your assessment, assign a depression level from Minimum to Severe.	Instruction: You will be given text data from a user's Reddit posts. Assume the role of a psychologist and classify the depression level of the user based on the text from the patient's Reddit posts. The depression level is on a scale of Minimum to Severe, where Minimum is no to minimal signs of depression, Mild is mild signs of depression, Moderate is moderate signs of depression, and Severe is very depressed. Here are some examples for context (delimited by double quotes):	Instruction: You are a psychologist analyzing text data from a user's reddit posts to determine their depression level. Classify the depression level of the user based on the text provided. The depression level is on a scale of Minimum to Severe, where: Minimum = No to minimal signs of depression Mild = Mild signs of depression Moderate = Moderate signs of depression Severe = Severe signs of depression Use the following chain of thought to analyze the text: 1. Identify any explicit statements related to feelings of sadness, hopelessness, failure, or unhappiness. 2. Assess the frequency and intensity of these feelings based on the user's descriptions. 3. Consider any additional context that might indicate a higher or lower depression level. 4. Based on your assessment, assign a depression level from Minimum to Severe. These are examples of your previous work:	Instruction: You will be given text data from a user. Assume the role of a psychologist and classify the depression level of the user based on the text from the patient's Reddit posts. The depression level is on a scale of Minimum to Severe, where Minimum is no to minimal signs of depression, Mild is mild signs of depression, Moderate is moderate signs of depression, and Severe is very depressed. Here are some examples for context (delimited by double quotes): (examples_str)
PromptPart2-Q	What is the depression level of this user?	Classify the following text:	What is the depression level of this user? Classify the following text:	What is the depression level of this user? Classify the following text:	What is the depression level of this user? Classify the following text:	Classify the following text:
OutputConstraint	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----	Format the output in JSON format. For example: {{ "depression_level": "Mild" }} ----

3.2 DSPy

Manual prompt engineering requires constant adjustment and testing for optimal performance, often necessitating several iterations. We explored a different paradigm to address these challenges using DSPy (Khattab et al., 2023), which is a Python framework that automates prompt and few-shot optimization.

The paradigm behind DSPy assumes that string-based prompts are verbose and prone to continuous refinement (Khattab et al., 2023). Therefore, it proposes that automatic optimization is more effective than manual optimization. DSPy has three key components: (a) Signatures, (b) Modules, and (c) Optimizers. Signatures are built upon pre-defined prompt templates provided by DSPy, where users specify the input and output variables of a given use case. Modules, also based on pre-defined prompts, represent established prompting techniques such as CoT and ReAct (Reason and Acting). Lastly, optimizers calibrate the signatures and modules by optimizing the prompt and/or selecting the optimal few-shot examples based on predefined metrics and nature of the task.

The following sections describe how each component was adapted for our specific use case.

Signatures: Input is defined as the text column of the dataset, and the output is the depression level of the text. Also provided is a brief prompt containing the depression scale (0 to 3) and a constraint i.e. label the text with within the scale (see [Appendix A](#) for an example). It highlights the pre-defined templates provided by DSPy, and the variables that we defined previously.

Modules: We used the *Predictor* module, which directly employs the signature and does not alter it, in contrast to other modules such as CoT, which elicit the LLM to reason before giving a label. The reason a *Predictor* module was used is to test the framework at its fundamental level, and to limit the token output.

Optimizers: Optimizers require three inputs: A predefined metric, a training dataset. and a module. The metric uses a simple boolean validation logic that checks if the prediction matches with the actual depression value of the post. The training set contains 80% of the depression dataset, comprising 2,886 rows, which the optimizer iterates over in several rounds using the metric as guidance.

We utilized the MiproV2 optimizer (Opsahl-Ong et al., 2024), which enhances performance through two key techniques: prompt optimization and bootstrapping. Bootstrapping involves sampling from the training set for few-shot application, conducted in rounds with a specified number of candidates. Finally, MiproV2 proposes an optimized prompt using a Bayesian surrogate model, which tests different combinations of sampled examples and prompt candidates to determine the optimal configuration.

The final prompt (see [Appendix B](#)), generated from automatically optimizing using MiproV2, incorporates the best combination of sampled example and proposed instructions, as determined by the Bayesian surrogate model. The optimization process parameters included 30 evaluation rounds and 20 candidates, following recommendations from the MiproV2 documentation.

3.3 RAG

While previous techniques ([Prompt Engineering](#) and [DSPy](#)) explore two different paradigms in prompt optimization, RAG represents a distinct approach that complements prompt engineering. RAG is defined as enhancing an LLM by integrating retrieved text passages from an external knowledge base (Jurafsky & Martin, 2023a). This grounds LLMs by providing contextual and relevant passages from

external information. In our approach, RAG was incorporated by enhancing few-shot prompting with retrieved text examples from the training dataset based on the similarity to the text input and depression level as metadata (see [Figure 4](#)). To test whether the number of examples influences the result, we specified the retriever to retrieve 1 or 2 examples based on relevancy.

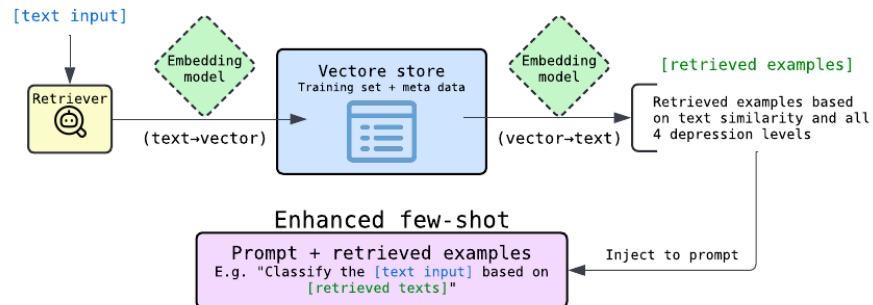


Figure 4 – RAG implementation

The above process was implemented using Langchain, which is framework in Python to create LLM-applications. Langchain offers functions and tools that facilitate building RAG applications, including data loading, text slicing, and retrieval, making the implementation convenient. The entire process of incorporating RAG for our use case can be split into four key steps:

3.3.1 Loading

Given that various data sources exist with differing formats and structures, correctly loading the data is a prerequisite for the retriever to function effectively. First, we used Langchain's CSV loader to load the training dataset in the right format so that it can be processed by the retriever. Metadata, which provides context to other data, is extracted from the "depression_level" column. Metadata can be used during the retrieval process to specify the criteria a text should fulfil to enhance the retrieval process and could be expanded to include more columns.

3.3.2 Text splitting

Splitting is the process of dividing a large text corpus into manageable chunks to store and retrieve from (Langchain, n.d.). For this data format, we did not find it necessary to further split the training set. The CSV already organize data into rows and columns, and using the entire text of a cell is more suitable for our application. Given that the data is sourced from Reddit, each row is a separate post. This allows for preserving the context and coherence inherent to the original posts, which makes the semantic similarity search more convenient.

3.3.3 Vector store and embedding

Embeddings transform texts into vectors (Langchain, n.d.). Our application finds similarities by comparing the input query with the encoded vectors. A vector store is a database that stores these embedded vectors. We used ChromaDB for the vector store, because of its integration with the Langchain framework, and OpenAIEmbeddings for the embedding, because we used OpenAI models. Therefore, we embedded the source column, then stored it in the vector store, while metadata was used as filter.

3.3.4 Retrieval

The retriever tool searches the vector store to return texts based on text similarity (Langchain, n.d.). We applied a filter using the ‘depression_level’, in which one example is pulled from each depression level. We specified that the retriever should return the top one or top two most similar texts. This approach allows us to measure whether increasing few-shot examples improves results. We then invoked the retriever and passed text inputs from the training set, allowing it to search through the ChromaDB vector store and retrieve the most relevant texts. RAG-enhanced few-shot prompting allows us to select examples that are highly relevant and contextually similar to the input query, rather than relying on random or one-size-fits-all examples (see [Prompt engineering](#)). Therefore, our RAG solution ensures that the application is grounded in pertinent examples to the input text.

3.4 Traditional models

Finally, we used NB and LR as benchmarks to contrast to previous LLM-based technology, and to evaluate how simpler models perform on the dataset.

The traditional models apply a different paradigm to LLMs. LLMs capture semantic meaning by embedding texts into high-dimensional vector space. In contrast, NB and LR rely on word count and statistical models. NB assumes that the features are conditionally independent given the class label, and that the position of features does not matter. Logistic Regression, on the other hand, predicts a linear relationship between the input features and the log-odds of the class probabilities. Both approaches are computationally low and cost-free, which we deemed as ideal for contrasting previous sections.

The implementation of NB and LR can be split into three stages: preprocessing, fit and transform, and training.

We employed under-sampling to account for the imbalance in the training set, which involves removing data from the majority label. This balanced the training set, resulting in more accurate performance measurements. For the fit and transformation, we used TF-IDF vectorizer to capture the importance of terms and frequently mitigating less-informative words (Jurafsky & Martin, 2023b). This approach contrasts with Bag-of-Words model, which only considers term frequency. Therefore, we argue TF-IDF is better suited when there is large amount of textual and verbose data as is our case.

Lastly, for the training, we used Multinomial NB (MNB) and LR in Python. MNB was used, because it is well-suited for text-classification tasks, as it handles discrete data, unlike Gaussian NB, which is more appropriate for continuous data distributions (Naulak & Jindal, 2022).

Part IV - Results

This section highlights the evaluation metrics that were selected as effective in assessing model performance. They were chosen from a more comprehensive list of investigated metrics (for the extended results see [Appendix D](#)); the main selection criteria were the dataset imbalance and the general scope of model application.

Table 5 - GPT-3.5 Turbo Models performance

		baseline	zero-shot		few-shot				DSPy	reference models	
			w/o CoT	CoT	w/o CoT	CoT	RAG w/1 ex.	RAG w/2 ex.		LR	NB
F1	macro	0,21	0,25	0,26	0,33	0,29	0,30	0,31	0,39	0,41	0,31
	micro	0,25	0,40	0,31	0,50	0,45	0,40	0,41	0,48	0,64	0,40
Accuracy	macro	0,31	0,35	0,32	0,40	0,35	0,40	0,41	0,35	0,46	0,42
	micro	0,22	0,34	0,26	0,43	0,36	0,34	0,34	0,57	0,59	0,35
MAE		1,16	1,09	1,31	0,83	0,96	0,83	0,82	0,73	0,80	1,33
RMSE		1,44	1,45	1,67	1,19	1,35	1,10	1,10	1,11	1,36	1,80

Table 6 - GPT 4o Models performance

		baseline	zero-shot		few-shot				DSPy	reference models	
			w/o CoT	CoT	w/o CoT	CoT	RAG w/1 ex.	RAG w/2 ex.		LR	NB
F1	macro	0,30	0,31	0,35	0,36	0,36	0,31	0,32	-	0,41	0,31
	micro	0,50	0,47	0,57	0,56	0,59	0,49	0,49	-	0,64	0,40
Accuracy	macro	0,36	0,40	0,42	0,43	0,43	0,37	0,39	-	0,46	0,42
	micro	0,42	0,41	0,50	0,49	0,52	0,41	0,41	-	0,59	0,35
MAE		0,95	0,97	0,82	0,80	0,76	0,88	0,89	-	0,80	1,33
RMSE		1,36	1,35	1,26	1,21	1,20	1,26	1,27	-	1,36	1,80

Given that the test set was proportionately generated from an imbalanced dataset, the imbalance persists within the test set. The main issue is the lack of evidence that the present distribution of depression is representative of either the general population, or any specific subset (e.g. Reddit users) that could be relevant to a specific use case. Therefore, the results from the test set as they stand do not necessarily reflect the classification of depression in a domain, general or specific, and thus, the imbalance has no use.

To counteract this, the focus shifts to the *macro* averages of the performance metrics which weigh all depression levels equally regardless of their prevalence in the dataset. Macro averages significantly reduce the weight of the majority class (minimal depression) and equally increase that of the other, minority classes since their populations are almost equal. This approach ensures a fair representation of all depression levels, not reliant on their arbitrary distribution in the data. However, it needs to be acknowledged that the confidence in the results for the majority class is higher compared to the minority classes because of the larger sample size.

The primary metric, chosen for its applicability to multiple different applications, is macro *F1*. *F1* strikes a balance between type I and type II errors, by being the harmonic mean of precision and recall. Therefore, it provides a useful metric in both situations where: misclassifying healthy people as depressed is costly (e.g. cases where the model is used for preliminary screenings - when psychologists may be misallocated); as well as situations where misclassifying depressed people as healthy is costly (e.g. cases where the model deals with high-risk populations; for example severe depression/self-harm). Thus, models are assessed by how well they serve a “generalist” role. In specific cases where the relative importance of type I and type II errors differ, the model assessment may also differ.

Albeit useful, the interpretability of F1 is somewhat limited. Thus, accuracy will be used as a supplementary metric, also completing the assessment by incorporating a measure involving true negatives⁶. Finally, since both above measures function in a “binary” fashion, not considering the distance between the predicted and actual value, MAE and RMSE complement the selected collection of metrics by quantifying the prediction error, with RMSE penalizing bigger misclassification distances.

Part V - Discussion

Among the GPT models, the best performing is the DSPy model (using GPT 3.5 Turbo). However, its performance is comparable to the LR model. In terms of macro F1, LR performs 5% better although the error (RMSE) of DSPy is almost 25% lower. This means that the LLM model misclassifies more datapoints compared to the traditional one; nonetheless, among the misclassified datapoints, the error of LR is notably higher. In other words, LR “wins more”, but DSPy “loses less”. Given the “understanding” of language by LLMs, in the data outliers, e.g. when a new word that is important in identifying depression is introduced (or when context is important), the chance of severe misclassification is lower compared to a non-semantic model. That is because in the semantic space of LLMs, the new word is already relatively close to depression whereas the non-semantic models cannot capture this relation since it is not connected to any depression level. Nevertheless, this could all the while prove detrimental to LLM-performance due to the difficulty of reestablishing the relation between the target word and depression compared to the traditional models that establish word-depression relationships solely based on the training text.

Table 7 - Impact of Applied Methodologies on Performance

	gpt 3.5	gpt 4o	avg
zero-shot vs. baseline			
macro F1	16,0%	3,2%	9,6%
RMSE	0,7%	-0,7%	0,0%
additional effect of CoT			
macro F1	3,2%	11,1%	7,1%
RMSE	13,1%	-7,2%	2,9%
few-shot vs. baseline			
macro F1	36,4%	16,7%	26,5%
RMSE	-21,0%	-13,3%	-17,2%
additional effect of CoT			
macro F1	-8,8%	0,0%	-4,4%
RMSE	14,3%	-0,9%	6,7%
RAG with 1 example vs. few shot			
macro F1	-6,4%	-13,4%	-9,9%
RMSE	-9,9%	-4,5%	-7,2%
additional effect of 2nd example			
macro F1	2,3%	3,0%	2,6%
RMSE	0,0%	-0,8%	-0,4%
DSPY vs. baseline			
macro F1	46,2%	-	46,2%
RMSE	-29,7%	-	-29,7%

Green numbers signify positive effect (increase in F1/decrease in error),
Red numbers signify a negative effect (decrease in F1/increase in error).

⁶ Cases where text is correctly not attributed to a wrong depression level.

Even if the performance of the best model is comparable to a much simpler model, our applied methodology managed to have a positive effect on performance compared to the baseline model. Manual prompt engineering (zero-shot) increased model performance (macro F1) by about 10% but had little effect on the error. Introducing relevant (handpicked) examples with few-shot significantly increased performance by 26% and reduced the error by 17%. Changing the examples to semantically similar posts by applying a RAG framework produced mixed results. Compared to few-shot, on one hand performance decreased (although still better than baseline). On the other hand, RAG also decreased the error. High text similarity appears to introduce noise, but also increase model reliability. The effect of the introduction of an additional example per depression level was marginal, yet positive. Finally, the introduction of CoT had mixed results. The overall effect on the 3.5 models is negative, while the positive effect on 4o is limited only when applied to zero-shot. Possibly the instructions were too complex for the older model to follow, whereas the newer model did not require more information or structure to “understand” the task.

However, in non-comparative, absolute terms, the performance of even the best models we researched is sub-par; with 40% of texts being misclassified. Therefore, the chosen models require a further performance increase to be used in realistic depression classification scenarios. The comparison of the results of our research to Xu et al. (2024) highlights another important issue with GPT models; the lack of control and transparency. The difference in performance, even in the baseline methods where replicating the methodology is trivial, can be most likely attributed to the model themselves; further reinforced by the result similarity of our limited GPT4 run, removing other possible factors such as the data split or pipeline. As indicated by Chen et al. (2023): “the behaviour of the “same” LLM service can change substantially in a relatively short amount of time”, even on the same task. Still, given the promising initial results for GPT 4, future research should investigate the efficiency of DSPy applied in conjunction to GPT 4 which we did not pursue due to high cost.

Finally, our research is severely limited due to the nature of the task itself. In a professional setting, depression is diagnosed using structured questionnaires (APA., 2013; Hamilton., 1960) that have the patient reflect on their thoughts and behaviour over a longer period. To transfer clinical depression diagnosis to Reddit posts becomes challenging, because even though the labelling methodology is based on the clinical depression standards, it was applied solely in Nasseem et al. (2022) and does not constitute “standard practice”. Therefore, possible shortcomings in the methodology would affect the results’ validity that reflect the “truth of the data” which does not necessarily provide an accurate depiction of reality.

Bibliography

All references were retrieved between June and July 2024.

- American Psychiatric Association. (2013). *Diagnostic and statistical manual of mental disorders* (5th ed.). <https://doi.org/10.1176/appi.books.9780890425596>
- Chen, L., Zaharia, M., & Zou, J. (2023). *How is ChatGPT’s behavior changing over time?* arXiv (Cornell University). <https://doi.org/10.48550/arxiv.2307.09009>
- Hamilton, M. (1960). *Hamilton Rating Scale for Depression (HRSD, HAM-D)* [Database record]. APA PsycTests. <https://doi.org/10.1037/t04100-000>
- IHME, Global Burden of Disease (2024) – with major processing by Our World in Data. “Depressive disorders prevalence” [dataset]. *IHME, Global Burden of Disease, “Global Burden of Disease - Mental Health Prevalence”* [original data]. Retrieved August 5, 2024 from <https://ourworldindata.org/grapher/depressive-disorders-prevalence-ihmejur>
- Jurafsky, D., & Martin, J. H. (2023a). Question Answering and Information Retrieval. In *Speech and Language Processing* (3rd ed. draft, pp. 294–314).

- Jurafsky, D., & Martin, J. H. (2023b). Vector Semantics and Embeddings. In *In Speech and Language Processing* (3rd ed. draft, pp. 105–135).
- Kailai Yang, Shaoxiong Ji, Tianlin Zhang, Qianqian Xie, and Sophia Ananiadou. 2023. On the Evaluations of ChatGPT and Emotion enhanced Prompting for Mental Health Analysis. <http://arxiv.org/abs/2304.03347>
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., & Potts, C. (2023). *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*. <https://arxiv.org/abs/2310.03714>
- Bishal Lamichhane. 2023. Evaluation of ChatGPT for NLP-based Mental Health Applications. <http://arxiv.org/abs/2303.15727>
- Langchain. (n.d). Conceptual guide. From Langchain: <https://python.langchain.com/v0.2/docs/concepts/>
- Mostafa M. Amin, Erik Cambria, and Björn W. Schuller. 2023. Will Affective Computing Emerge from Foundation Models and General AI? A First Evaluation on ChatGPT. <http://arxiv.org/abs/2303.03186>
- Naseem, U., Dunn, A. G., Kim, J., & Khushi, M. (2022). *Early Identification of Depression Severity Levels on Reddit Using Ordinal Classification*. Proceedings of the ACM Web Conference 2022. <https://doi.org/10.1145/3485447.3512128>
- Naulak, C., & Jindal, P. R. (2022). A comparative study of Naive Bayes Classifiers with improved technique on Text Classification. *TechRxiv*.
- National Institute of Mental Health. (2024). Depression (NIH Publication No. 24-MH-8079). U.S. Department of Health and Human Services, National Institutes of Health. Retrieved [2024], from <https://www.nimh.nih.gov/health/publications/depression>
- Opsahl-Ong, K., Ryan, M. J., Purtell, J., Broman, D., Potts, C., Zaharia, M., & Khattab, O. (2024). *Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs*. <https://arxiv.org/abs/2406.11695>
- OpenAI. (n.d.). Models. OpenAI. Retrieved from <https://platform.openai.com/docs/models>
- OpenAI. (n.d.). Models. OpenAI. Retrieved from <https://platform.openai.com/docs/guides/batch/overview>
- Pennebaker, J.W., Boyd, R.L., Jordan, K., & Blackburn, K. (2015). *The development and psychometric properties of LIWC2015*. Austin, TX: University of Texas at Austin
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving language understanding by generative pre-training*.
- Rosendahl, H., Davidsen, M., Møller, S. R., Ibáñez Román, J. E., Kragelund, K., Christensen, A. I., & Ekholm, O. (2022). *Danskernes sundhed: Den Nationale Sundhedsprofil 2021*. Sundhedsstyrelsen. https://www.sst.dk/-/media/Udgivelser/2022/Sundhedsprofil/Sundhedsprofilen.ashx?sc_lang=da&hash=5C9A9A81483F6C987D5651976B72ECB2
- Turcan, E., & McKeown, K. (2019). *Dreaddit: A Reddit Dataset for Stress Analysis in Social Media*. Proceedings of the Tenth International Workshop on Health Text Mining and Information Analysis (LOUHI 2019). <https://doi.org/10.18653/v1/d19-6213>
- Xu, X., Yao, B., Dong, Y., Gabriel, S., Yu, H., Hendler, J., Ghassemi, M., Dey, A. K., & Wang, D. (2024). *Mental-LLM: Leveraging Large Language Models for Mental Health Prediction via Online Text Data*. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 8(1), 1–32. <https://doi.org/10.1145/3643540>

Appendix

Appendix A - DSPy Signature

The proceeding prompt is an example of a DSPy signature. The highlighted text is provided by us, which includes a brief prompt on the depression scale and the text and label as input and output. The overall format is provided by the framework.

Label the text from a scale from 0 to 3. 0 indicates minimum depression, 1 indicates mild depression, 2 indicates moderate depression and 3 indicates severe depression.

Follow the following format.

Text: \${text}

Label: 0, 1, 2, or 3

Appendix B - DSPy optimized prompt

Analyze the given text thoroughly to assess the level of depression present, categorizing it on a scale from 0 to 3. Assign a label of 0 for minimum depression, 1 for mild depression, 2 for moderate depression, and 3 for severe depression based on the emotional content and sentiment expressed in the text.

Follow the following format.

Text:

\${text}

Label: 0, 1, 2, or 3

Text: Recently my anxiety has gotten really bad, possibly the worse it has ever been and I've been experiencing panic attacks, but I still can't justify taking my medication because I feel like I deserve to suffer. I also struggle with the feeling like I'm making all this up and none of my problems exist which I know rationally cannot be true when I've struggled with this for most my life, however I still can't shift it. I don't know if this is part of my anxiety making me feel like this or whether it is another thing I've made myself believe. I guess I'm just looking for some advice and reassurance. I'm just really tired of feeling this way and constantly being in turmoil with my mind. Label: 3

Appendix C - Additional Testing

Prompt engineering was applied iteratively, with numerous tested formats outside the aforementioned discarded due to low performance:

- 1) GPT 3.5 and 4o had different problems (incl. the calculation of performance metrics) with the output until JSON formatting was introduced.
- 2) The verbosity of the prompt was tested across numerous formats, showing that clear and concise directions performed the best, as opposed to overly complicated directions.
- 3) The classification scale was also tested in numerical and verbal form, with the latter being chosen as to not "confuse" the model. This is due to the "no to mild symptoms" class being labelled as 0, which might lead to the model assigning only text with no depression levels to this class.
- 4) Context enrichment was also tested in several formats. The *origin of the text* enhancement was specified as coming from the patient's social media, or Reddit posts, with no significant performance differences. Moreover, the *role* enrichment was tested as psychologist, mental health professional, and

mental health professional specializing in depression, with the psychologist role being chosen to reduce verbosity.

5) Finally, we tested asking the model to provide the rationale prior to classifying the text, which reduced performance, while also having JSON format problems and a high token cost.

All the initial prompt development was performed on the validation set, apart from the RAG prompts, which required specific preprocessing to develop a dataset with the most relevant examples per case.

Appendix D - Extended Results

GPT 4o

Results for prompt type: baseline

Accuracy: 41.71%
Mean Absolute Error (MAE): 0.9558011049723757
Mean Squared Error (MSE): 1.856353591160221
Root Mean Squared Error (RMSE): 1.3624806755180863
Confusion Matrix:
[[118 55 65 27]
[1 4 15 9]
[1 8 18 12]
[1 4 13 11]]
Precision Score: 0.34503686996688143
Recall Score: 0.3560157149291827
F1 Score: 0.29534974093264243
Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.45	0.61	265
Class 1	0.06	0.14	0.08	29
Class 2	0.16	0.46	0.24	39
Class 3	0.19	0.38	0.25	29

accuracy 0.42 362
macro avg 0.35 0.36 0.30 362
weighted avg 0.75 0.42 0.50 362

Results for prompt type: zero-shot

Accuracy: 40.56%
Mean Absolute Error (MAE): 0.9777777777777777
Mean Squared Error (MSE): 1.8333333333333333
Root Mean Squared Error (RMSE): 1.35400640077266
Confusion Matrix:
[[103 50 95 15]
[1 3 19 6]
[1 5 30 3]
[1 4 14 10]]
Precision Score: 0.3760190686905209
Recall Score: 0.402285403072082
F1 Score: 0.31155712349224796
Classification Report:

	precision	recall	f1-score	support
Class 0	0.97	0.39	0.56	263
Class 1	0.05	0.10	0.07	29
Class 2	0.19	0.77	0.30	39
Class 3	0.29	0.34	0.32	29

accuracy 0.41 360
macro avg 0.38 0.40 0.31 360
weighted avg 0.76 0.41 0.47 360

Results for prompt type: zero-shot chain of thought

Accuracy: 50.28%
Mean Absolute Error (MAE): 0.8204419889502762
Mean Squared Error (MSE): 1.5883977900552486
Root Mean Squared Error (RMSE): 1.2603165435934136
Confusion Matrix:
[[143 47 54 21]
[1 4 15 9]
[3 6 24 6]
[1 7 10 11]]
Precision Score: 0.3739421195363924
Recall Score: 0.41806215905109856
F1 Score: 0.35150432633304746
Classification Report:

	precision	recall	f1-score	support
Class 0	0.97	0.54	0.69	265
Class 1	0.06	0.14	0.09	29
Class 2	0.23	0.62	0.34	39
Class 3	0.23	0.38	0.29	29

accuracy 0.50 362
macro avg 0.37 0.42 0.35 362
weighted avg 0.76 0.50 0.57 362

Results for prompt type: few-shot

Accuracy: 49.17%
Mean Absolute Error (MAE): 0.8
Mean Squared Error (MSE): 1.4722222222222223
Root Mean Squared Error (RMSE): 1.2133516482134197
Confusion Matrix:
[[134 54 60 15]
[2 5 16 6]
[2 6 29 2]
[1 5 14 9]]
Precision Score: 0.39010120684964633
Recall Score: 0.433963516925363
F1 Score: 0.3574618356212048
Classification Report:

	precision	recall	f1-score	support
Class 0	0.96	0.51	0.67	263
Class 1	0.07	0.17	0.10	29
Class 2	0.24	0.74	0.37	39
Class 3	0.28	0.31	0.30	29

accuracy 0.49 360
macro avg 0.39 0.43 0.36 360
weighted avg 0.76 0.49 0.56 360

Results for prompt type: few-shot chain of thought

Accuracy: 52.49%
Mean Absolute Error (MAE): 0.7679558011049724

Mean Squared Error (MSE): 1.4585635359116023
 Root Mean Squared Error (RMSE): 1.2077100380106154
 Confusion Matrix:
 [[150 45 52 18]
 [3 4 13 9]
 [2 5 27 5]
 [1 5 14 9]]
 Precision Score: 0.37589106199049493
 Recall Score: 0.4266553225564286
 F1 Score: 0.3582637036974001
 Classification Report:

	precision	recall	f1-score	support
Class 0	0.96	0.57	0.71	265
Class 1	0.07	0.14	0.09	29
Class 2	0.25	0.69	0.37	39
Class 3	0.22	0.31	0.26	29

	accuracy			
macro avg	0.38	0.43	0.36	362
weighted avg	0.75	0.52	0.59	362

Results for prompt type: RAG One

Accuracy: 41.00%
 Mean Absolute Error (MAE): 0.8864265927977839
 Mean Squared Error (MSE): 1.5955678670360112
 Root Mean Squared Error (RMSE): 1.263157894736842
 Confusion Matrix:
 [[112 80 52 20]
 [1 6 18 4]
 [3 8 22 6]
 [1 6 14 8]]
 Precision Score: 0.3588346107164379
 Recall Score: 0.36777590225866086
 F1 Score: 0.30580100275290695
 Classification Report:

	precision	recall	f1-score	support
Class 0	0.96	0.42	0.59	264
Class 1	0.06	0.21	0.09	29
Class 2	0.21	0.56	0.30	39
Class 3	0.21	0.28	0.24	29

	accuracy			
macro avg	0.36	0.37	0.31	361
weighted avg	0.74	0.41	0.49	361

Results for prompt type: RAG Two

Accuracy: 41.44%
 Mean Absolute Error (MAE): 0.8922651933701657
 Mean Squared Error (MSE): 1.6215469613259668
 Root Mean Squared Error (RMSE): 1.2733997649308588
 Confusion Matrix:
 [[110 80 55 20]
 [1 6 18 4]
 [3 6 25 5]
 [1 7 12 9]]
 Precision Score: 0.37031065806809516
 Recall Score: 0.39334033998965684
 F1 Score: 0.31923113865940517
 Classification Report:

	precision	recall	f1-score	support
Class 0	0.96	0.42	0.58	265
Class 1	0.06	0.21	0.09	29
Class 2	0.23	0.64	0.34	39
Class 3	0.24	0.31	0.27	29

	accuracy			
macro avg	0.37	0.39	0.32	362
weighted avg	0.75	0.41	0.49	362

GPT 3.5

Results for prompt type: baseline

Accuracy: 21.73%
 Mean Absolute Error (MAE): 1.1615598885793872
 Mean Squared Error (MSE): 2.0807799442896937
 Root Mean Squared Error (RMSE): 1.442490881874022
 Confusion Matrix:
 [[42 129 62 29]
 [0 8 13 8]
 [1 10 18 10]
 [0 7 12 10]]
 Precision Score: 0.34388985147859075
 Recall Score: 0.3106333650555814
 F1 Score: 0.21134991739738213
 Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.16	0.28	262
Class 1	0.05	0.28	0.09	29
Class 2	0.17	0.46	0.25	39
Class 3	0.18	0.34	0.23	29

	accuracy			
macro avg	0.34	0.31	0.21	359
weighted avg	0.75	0.22	0.25	359

Results for prompt type: zero-shot

Accuracy: 34.25%
 Mean Absolute Error (MAE): 1.0994475138121547
 Mean Squared Error (MSE): 2.116022099447514
 Root Mean Squared Error (RMSE): 1.4546553198086183
 Confusion Matrix:
 [[85 56 100 24]
 [1 1 20 7]
 [1 2 30 6]
 [0 4 17 8]]
 Precision Score: 0.3375757516166354
 Recall Score: 0.35008257844952706
 F1 Score: 0.2530430070069686
 Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.32	0.48	265
Class 1	0.02	0.03	0.02	29
Class 2	0.18	0.77	0.29	39
Class 3	0.18	0.28	0.22	29

	accuracy			
macro avg	0.34	0.35	0.25	362
weighted avg	0.75	0.34	0.40	362

Results for prompt type: few-shot

Accuracy: 42.54%
 Mean Absolute Error (MAE): 0.8342541436464088
 Mean Squared Error (MSE): 1.430939226519337
 Root Mean Squared Error (RMSE): 1.1962187201842884
 Confusion Matrix:
 [[113 86 52 14]
 [1 8 18 2]
 [1 11 26 1]
 [0 11 11 7]]
 Precision Score: 0.39655788344145687
 Recall Score: 0.40258078507915857
 F1 Score: 0.3313492491623351

Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.43	0.59	265
Class 1	0.07	0.28	0.11	29
Class 2	0.24	0.67	0.36	39
Class 3	0.29	0.24	0.26	29
accuracy		0.43	362	
macro avg	0.40	0.40	0.33	362
weighted avg	0.77	0.43	0.50	362

Results for prompt type: zero-shot chain of thought

Accuracy: 26.32%

Mean Absolute Error (MAE): 1.3128654970760234

Mean Squared Error (MSE): 2.8157894736842106

Root Mean Squared Error (RMSE): 1.67803142809788

Confusion Matrix:

[[55 69 62 60]

[0 2 15 12]

[0 3 16 19]

[0 3 9 17]]

Precision Score: 0.3350610446198682

Recall Score: 0.3249505702861021

F1 Score: 0.2199827412732932

Classification Report:

	precision	recall	f1-score	support
Class 0	1.00	0.22	0.37	246
Class 1	0.03	0.07	0.04	29
Class 2	0.16	0.42	0.23	38
Class 3	0.16	0.59	0.25	29
accuracy		0.26	342	
macro avg	0.34	0.32	0.22	342
weighted avg	0.75	0.26	0.31	342

Results for prompt type: few-shot chain of thought

Accuracy: 36.19%

Mean Absolute Error (MAE): 0.9668508287292817

Mean Squared Error (MSE): 1.828729281767956

Root Mean Squared Error (RMSE):

1.352305173312576

Confusion Matrix:

[[98 104 26 37]

[1 9 10 9]

[1 14 11 13]

[0 9 7 13]]

Precision Score: 0.35760893246187364

Recall Score: 0.35262082311529286

F1 Score: 0.28501552320499396

Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.37	0.54	265
Class 1	0.07	0.31	0.11	29
Class 2	0.20	0.28	0.24	39
Class 3	0.18	0.45	0.26	29
accuracy		0.36	362	
macro avg	0.36	0.35	0.29	362
weighted avg	0.76	0.36	0.45	362

Results for prompt type: RAG One

Accuracy: 33.70%

Mean Absolute Error (MAE): 0.8314917127071824

Mean Squared Error (MSE): 1.223756906077348

Root Mean Squared Error (RMSE):

1.1062354659281848

Confusion Matrix:

[[79 151 25 10]

[0 19 9 1]

[2 16 19 2]

[0 13 11 5]]

Precision Score: 0.4113597016719399

Recall Score: 0.4032197254058022

F1 Score: 0.3012530154480893

Classification Report:

	precision	recall	f1-score	support
Class 0	0.98	0.30	0.46	265
Class 1	0.10	0.66	0.17	29
Class 2	0.30	0.49	0.37	39
Class 3	0.28	0.17	0.21	29
accuracy		0.34	362	
macro avg	0.41	0.40	0.30	362
weighted avg	0.78	0.34	0.40	362

Results for prompt type: RAG Two

Accuracy: 33.98%

Mean Absolute Error (MAE): 0.8259668508287292

Mean Squared Error (MSE): 1.212707182320442

Root Mean Squared Error (RMSE):

1.1012298499043884

Confusion Matrix:

[[78 152 25 10]

[0 19 9 1]

[1 15 21 2]

[0 13 11 5]]

Precision Score: 0.41969468876154203

Recall Score: 0.41509684199989993

F1 Score: 0.3082302490516246

Classification Report:

	precision	recall	f1-score	support
Class 0	0.99	0.29	0.45	265
Class 1	0.10	0.66	0.17	29
Class 2	0.32	0.54	0.40	39
Class 3	0.28	0.17	0.21	29
accuracy		0.34	362	
macro avg	0.42	0.42	0.31	362
weighted avg	0.79	0.34	0.	362